

Structures de données

Interface et implémentation

1. Idée générale

Un programme manipule des **données** — des nombres, des textes, des collections d'objets. Pour les organiser efficacement, on les range dans des **structures de données**.

Une structure de données a toujours deux aspects bien distincts :

- son **interface** : ce qu'elle sait faire, vu de *l'extérieur* — les opérations disponibles, ce qu'elles prennent en entrée, ce qu'elles renvoient ;
- son **implémentation** : comment elle le fait *concrètement*, en mémoire et en code.

Exemple : analogie avec le monde sensible ▼

Une télécommande a une **interface** : des boutons avec des fonctions connues (Volume +, Muet ...). Son **implémentation**, c'est l'électronique intérieure — on ne la voit pas, et on n'a pas besoin de la connaître pour utiliser la télécommande. On pourrait remplacer les circuits par d'autres, sans changer les boutons : l'interface resterait identique.

Bénéfice

Cette séparation interface / implémentation est **fondamentale**, en informatique. Elle permet de **changer l'implémentation sans modifier le reste du programme**. Un développeur peut donc *complètement refondre* le code implémentant une fonctionnalité *sans impacter un autre programmeur qui l'utilise* — tant que l'interface est respectée (et le code fonctionnel, bien sûr).

2. Interface : spécifier sans implémenter

Spécifier une structure, c'est décrire son interface. On doit nommer et décrire explicitement les opérations qu'elle offre, mais sans expliquer *comment* elles fonctionnent.

Imaginons une structure « pile », qui empile les données comme on empile des assiettes dans un placard :

- une nouvelle entrée est placée au sommet de la pile — c'est la *dernière* arrivée ;
- c'est aussi la *première* à partir, car on retire les assiettes en partant du sommet de la pile.

Son interface est généralement spécifiée ainsi :

Opération	Description
<code>empiler(pile, valeur)</code>	Ajoute valeur au sommet de la pile
<code>depiler(pile)</code>	Retire et renvoie la valeur au sommet
<code>est_vide(pile)</code>	Renvoie True si la pile est vide
<code>sommet(pile)</code>	Consulte la valeur au sommet sans la retirer

Cette spécification ne dit *rien* sur la façon dont la pile est stockée en mémoire : c'est *volontaire*.

 **Attention**

En NSI, « liste », « pile » et « file » désignent des **structures abstraites** (des interfaces). Ne pas les confondre avec la **liste Python** (type `list`), qui est une implémentation *concrète particulière*. Ce point est développé dans une autre fiche (à consulter, évidemment).

3. Implémentation : concrétiser l'interface

Une même interface peut être traduite en code informatique par des implémentations très différentes. C'est tout l'intérêt de la séparation : on peut *choisir* l'implémentation la plus adaptée à la situation.

Deux critères guident ce choix :

- la **complexité** des opérations (certaines implémentations rendent une opération plus rapide qu'une autre) ;
- les **contraintes** du problème (mémoire disponible, type de données, fréquence des opérations...).

4. Le cas particulier du « tableau » en Python

4.1. Définition du tableau

Un **tableau** est :

- une suite d'un nombre *fixé à l'avance* d'éléments de *même type* — ce qui garantit que chaque élément occupe exactement le *même espace en mémoire* ;
- ces éléments sont *numérotés* par un *indice* (un entier, généralement compté à partir de `0`) ;
- ils sont donc rangés dans un *ordre fixe* mais surtout placés de façon *contiguë en mémoire*.

De cette façon, un élément du tableau est accessible *directement* depuis son indice *i*. Il suffit en effet de savoir *où* débute le tableau, dans la mémoire vive de l'ordinateur. Notons *m* cette adresse mémoire. Connaissant la taille *n* commune à tous les éléments du tableau, l'élément d'indice *i* sera à l'adresse $m + i \times n$. L'accès et la modification d'un élément du tableau sont donc *très efficaces*, et **ne** dépendent **pas** de la taille du tableau !

Son interface est minimaliste :

Opération	Description
<code>lire(tableau, i)</code>	Renvoie l'élément à l'indice <code>i</code>
<code>ecrire(tableau, i, valeur)</code>	Modifie l'élément à l'indice <code>i</code>
<code>taille(tableau)</code>	Renvoie le nombre d'éléments

 **Propriété intéressante et limite d'un tableau**

- Les opérations `lire` et `ecrire` d'un tableau s'effectuent *en temps constant* (on écrit « en $\mathcal{O}(1)$ »). Cela signifie que, *quel que soit l'indice `i` et quelle que soit la taille de `tableau`*, `lire(tableau, i)` et `ecrire(tableau, i, valeur)` ont une durée d'exécution identique.
- En revanche, un tableau a une **taille fixe** : on ne peut pas y ajouter ou supprimer des éléments. Et, s'il le faut vraiment, la création d'un *nouveau* tableau (plus grand) et la *recopie des éléments de l'ancien vers le nouveau* seront nécessaires.

Remarque : notation `tableau[i]` ▼

Il est d'usage d'abréger `lire(tableau, i)` en `tableau[i]`. De même, on écrit souvent `tableau[i] = valeur` au lieu de `ecrire(tableau, i, valeur)`.

Remarque : temps constant, ou à peu près constant? ▼

En théorie, on parle de *temps constant*. En pratique, il ne l'est jamais tout à fait. En effet, il y a toujours une *petite variabilité* des durées effectivement mesurées, lorsqu'on reproduit un grand nombre de fois une opération. Elle peut être due à la charge du processeur à ce moment, ou à une interruption du système, pour un accès au disque dur par exemple.

4.2. En Python

Python ne dispose pas d'un type « tableau » au sens strict. La **liste Python** (type `list`) joue ce rôle :

- elle offre l'accès par indice (en temps constant : c'est la complexité optimale pour cette opération) ;
- mais elle est en réalité un **tableau dynamique** : elle se redimensionne *automatiquement* lorsqu'on ajoute des éléments.

C'est une implémentation concrète, parmi d'autres, de la notion abstraite de *séquence indexée*.

Remarque : pourquoi « dynamique » ? ▼

Un tableau au sens *strict* a une taille fixée à la création. Un tableau *dynamique* peut grandir : lorsqu'il est plein, Python alloue un nouveau bloc mémoire plus grand et y recopie tous les éléments.

Pour aller plus loin

Ce redimensionnement est rare et « amorti » : en pratique, Python pré-alloue systématiquement *plus d'espace que nécessaire* pour une liste. Il consomme ainsi plus d'espace mémoire, au bénéfice du temps d'exécution : tant que l'espace alloué par Python à la liste n'est pas complètement utilisé, l'ajout de nouveaux éléments est très rapide. Ainsi, `list.append()` reste en moyenne en $\mathcal{O}(1)$.

5. Ce qu'il faut retenir

✓ Récapitulatif

- Une structure de données a une **interface** (ce qu'elle fait) et une **implémentation** (comment elle le fait).
- Spécifier, c'est décrire l'interface sans contraindre l'implémentation.
- Une même interface peut avoir plusieurs implémentations aux performances différentes.
- En Python, `list` joue le rôle de tableau dynamique — c'est une implémentation, pas une structure abstraite.